Redrob-Data-AI-Challenge

A blazing-fast, CPU-optimized NLP pipeline designed to screen hundreds of candidates instantly without AI hallucinations or heavy GPU dependencies.

Project Leader: Shreya Gupta

Solution Overview

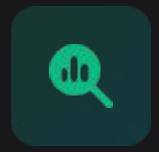
What We Built

We developed a lightweight, CPU-optimized Natural Language Processing pipeline. It intelligently parses candidate profiles, scores semantic matches against the Job Description (JD), and generates fact-based justifications for the top 100 candidates—all in mere seconds.

Why This Approach?

By relying on strict deterministic logic rather than generative LLMs, we guarantee **Zero Hallucinations**. Our system artificially "boosts" explicitly listed skills by doubling their weight, prioritizing verified knowledge over conversational fluff.

JD Understanding & Evaluation

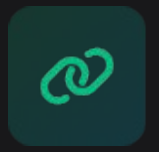


Keyword Foundation

We dynamically extract all meaningful vocabulary (words with 3+ characters) from the raw JD text to build a robust core foundation that matches strictly against actual candidate skills.

Signal Weighting

We analyze the whole picture—headline, summary, and past roles. Crucially, we apply double weight to explicit skill tags, preventing narrative manipulation.



Bi-Gram Context

Instead of isolated words, our system uses bi-grams to comprehend context. It recognizes "machine learning" as a specific skill phrase, not just the isolated words "machine" and "learning."

Ranking Methodology

Data Ingestion

The system reads raw JSONL data, cleans, and merges text, mapping both JD and candidates into a shared multidimensional space.

Matrix Math

We utilize TF-IDF Vectorization (capped at 20,000 features) combined with Cosine Similarity for precise semantic overlap scoring.

Strict Sorting

Candidates are strictly evaluated and ranked by their mathematical similarity score (ranging from 0.0 to 1.0) against the parsed Job Description.

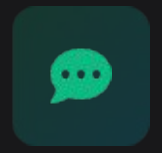
Tie-Breaking

To guarantee a perfect Top 100 list with absolutely no overlap, tied scores are deterministically broken using unique Candidate IDs.

Explainability & Validation

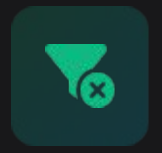
Zero Hallucinations

A strictly rule-based formatting engine writes justifications. It only outputs skills genuinely and explicitly extracted from the candidate's verified profile data.



Custom Justifications

Generates a concise 1-2 sentence explanation explicitly calling out Years of Experience (YoE) and highlighting the top 3 specific matching skills.



Spam Filtration

Profiles with inconsistent data, spam text, or missing histories naturally receive minimal Cosine Similarity scores and sink directly to the bottom of the rankings.

End-to-End Workflow

1

Ingest & Extract

Load JD text, parse candidate JSONL, and pull clean histories. Explicit skills are boosted 2x.

2

Vectorize

Convert all aggregated text into TF-IDF bi-gram matrices, creating a mathematical vocabulary.

3

Score & Rank

Calculate Cosine Similarity between the JD and candidates. Sort descending for the Top 100.

4

Reason & Export

Cross-reference skills for factual justification generation, then export to a final CSV.

System Architecture

Data Layer

Ingests raw inputs directly from job_description.txt and candidates.jsonl.

Processing Layer

Utilizes Python & Pandas for rapid data cleaning, string manipulation, sorting, and explicit skill boosting.

Math / NLP Engine

Powered by Scikit-Learn (TfidfVectorizer + cosine_similarity) mapping text to high-dimensional space.

Explainability Module

A strict rule-based logic engine that cross-references candidate capabilities (YoE + top skills).

Results & Performance

0

GPUs Required

Because it relies on Scikit-Learn's highly optimized C-backend operations rather than massive Deep Learning models, the pipeline processes thousands of profiles in seconds on an off-the-shelf CPU.

100%

Deterministic Ranking

By capturing two-word phrases and artificially boosting explicit skills, the system effectively ignores "fluff" and surfaces only the candidates who genuinely possess the required technical capabilities.

Technologies Used

Python

The core language, chosen for its unparalleled readability, speed in prototyping, and massive data processing ecosystem.

Pandas

The backbone of our processing layer. Used for blazing-fast data manipulation, row aggregation, sorting, and tie-breaking.

Scikit-Learn

Provides robust algorithms (TF-IDF & Cosine Similarity) for fast, mathematically sound text comparisons without Deep Learning overhead.

JSON & Regex

Standard libraries used for ultra-lightweight, dependency-free text parsing, data ingestion, and precision keyword extraction.

Submission Assets

Thank you for reviewing our lightweight, hallucination-free screening pipeline.

Submission Assets

Thank you for reviewing our lightweight,
hallucination-free screening pipeline.

GitHub:
https://github.com/Shreya-Gupta-5074/Redrob-Data-AI-Challenge

Demo:
https://drive.google.com/file/d/1SJMEZcsLEKaTbsZT0xjTIJuHW_pu7NnX/view?usp=sharing